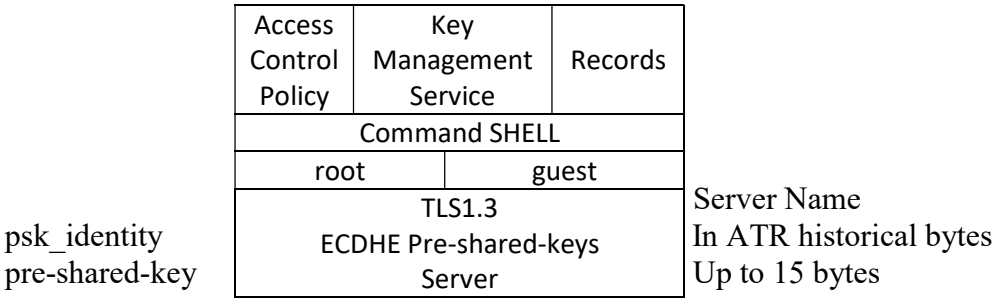
TLS-SE 3.03
SHELL

CONTENTS

About TLS for Secure Element Version 3.0.3

TLS-SE version 3.0.3 (TLS for Secure Element) is a Java Card 3.0.4/3.0.5 application designed for personal Network HSM (pNHSM) servers.



TLS-SE 3.0.3

TLS-SE is a hardware security module accessed through shell commands over a protected TLS 1.3 connection.

Two accounts are available: root and guest. They use different PSK identities and pre-shared keys. The root account can enable or disable cryptographic features.

The embedded TLS 1.3 server uses pre-shared key (PSK) authentication, a mode resistant to quantum attacks, with an Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) key exchange over the SECP256 prime curve.

A TLS-SE app is associated with a server name stored in the ATR (Answer to Reset) historical bytes (up to 15 bytes).

A TLS-SE app has a public/private key pair (on the SECP256k1 curve) and a certificate.
The Application Certification Procedure (ACP) verifies that the application is genuine.

TLS-SE provides two kinds of cryptographic services: KEYS associated with persistent cryptographic objects and SECRETS associated with on-demand cryptographic objects.

TLS-SE supports the following cryptographic procedures:

- RSA-2048 with PKCS #1 v1.5.
- EC-256 (SECP256p and SECP256k1) with ECDSA signatures.
- AES-128 encryption and decryption.
- HMAC-SHA-256.
- HMAC-SHA-512.
- BIP32 with hardened keys.
- TLS 1.3 PSK binder and key derivation procedures.

Two AES-128 wrap keys are available. Key blobs are built using AES-CCM authenticated encryption with associated data (AEAD) and a 16-byte tag.

TLS-SE provides four large files (up to 1,280 bytes) that can be used to store certificates and four small files (up to 64 bytes) that can be used to store messages.

TLS-SE Components

Keys
Four RSA-2048 keys.
Four EC-256 keys for the SECP256prime and SECP256k1 curves.

Secrets
Four secrets used for AES-128, HMAC-SHA-256, HMAC-SHA-512, and BIP32.

Wrap Keys
Two AES-128 wrap keys.

Export/Import Keys
All keys and secret values can be exported or imported as cryptographic blobs built using AES-CCM authenticated encryption with associated data (AEAD) and a 16-byte tag.

Files
Four large files up to 1,280 bytes.
Four small files up to 64 bytes.

Performance
Performance is measured over an end-to-end TLS session using a local (127.0.0.1) IoSE server and a J3R180 Java Card:

- TLS 1.3 session establishment: < 1,000 ms
- TLS 1.3 1,000-byte echo: < 1,000 ms
- ECDSA signature: 128 ms (x8)
- RSA-2048 signature: 300 ms (x4)
- HMAC-SHA-256: 75 ms (x16)
- HMAC-SHA-512: 100 ms (x16)
- AES-128: 32 ms (x32)

TLS-SE Command Line with OpenSSL

Use the following command:
openssl s_client -tls1_3 -connect IP:PORT -servername [**SecureElementName**]
-psk [**pre-shared-key**] -psk_identity [**psk-identity**] -groups P-256 -cipher DHE
-ciphersuites TLS_AES_128_CCM_SHA256 -no_ticket

Example
openssl s_client -tls1_3 -connect 127.0.0.1:8888 -servername **key1.com**
-psk **0102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F20**
-psk_identity **Client_identity** -groups P-256 -cipher DHE
-ciphersuites TLS_AES_128_CCM_SHA256 -no_ticket

Command Formats

A request contains a three-character ASCII header and a payload, followed by carriage-return (CR) and line-feed (LF) characters.
The payload format is text or binary.

Prefix: one character	Index: two hex digits	Payload (text or binary)	CR	LF

Request

In text format, CR may be omitted in a UNIX environment. In binary format, however, if the last payload byte is 0x0D, an additional CR character MUST precede LF.

A response contains a payload, followed by carriage-return (CR) and line-feed (LF) characters.

Payload (text or binary)	CR	LF

Response

A severe error is indicated by the TLS session closing; no response is returned.

Text Format

Request

Prefix: 1 character; index: 2 hexadecimal digits (0123456789ABCDEF); payload (up to 1,280 characters); CR LF

Response

Payload (up to 1,280 characters); CR LF
or
ERROR CR LF

Binary Format

Request

Prefix: 1 character; index: 2 hexadecimal digits (0123456789ABCDEF); payload (up to 1,280 bytes); CR LF

Response

Payload (up to 1,280 bytes); CR LF
or
Disconnection

Command Index Encoding

Commands are identified by a character prefix and divided into three logical groups: administrative (Admin), cryptographic (Crypto), and read/write (RW) commands.

Every command is associated with an index encoded as two hexadecimal digits.

Administrative Command

An Admin command uses the '?' prefix. It is identified by a two-hexadecimal-digit index representing an integer from 0 to 255.

Cryptographic Command

A cryptographic command has a character prefix. Its index, called KeyIndex, comprises a key identifier (KeyId) and an algorithm identifier (AlgoId).
KeyIndex is a byte denoted by $b_7b_6b_5b_4b_3b_2b_1b_0$. It is divided into three fields: Mode, AlgoId, and KeyId

b_7b_6: Mode
b_7: 1 = binary command; 0 = text command
b_6: 1 = multiple input; 0 = single command

$b_5b_4b_3b_2$: AlgoId
0000 = EC256P or implicit; 0100 = EC256k1; 1000 = RSA2048; 1100 = Secret; 1100 = BIP32.
When the cryptographic procedure is identified by the command prefix, AlgoId is ignored and MAY be set to zero.

b_1b_0: KeyId
Values range from 0 to 3.

Read/Write Command

For an RW command, the index is a record identifier (RecId) ranging from 0 to 7.

$b_2b_1b_0$: RecId
For records 0 to 3, the maximum size is 1,280 bytes.
For records 3 to 7, the maximum size is 64 bytes.

Blobs and Wrap Keys

Two wrap keys are available; both use AES-128. Blobs are built using AES-CCM authenticated encryption with associated data (AEAD) and a 16-byte tag.

Blob Format

A blob stores an encrypted key or secret value. Its contents are divided into seven parts: BlobType, KeyIndex, KeyParam, BlobId, BlobRnd, EncryptedValue, and Tag, as shown below.

Blob: BlobType (1B) || KeyIndex (1B) || KeyParam (1B) || BlobId (4B) || BlobRnd (12B) || EncryptedValue || Tag (16B)

BlobType
BlobType identifies the blob version. It is set to zero for this version.

KeyIndex
KeyIndex is the concatenation of AlgoId (4 bits) and KeyId (xy = 2 bits); the two most significant bits are set to zero.
00 0000 xy: SECP256Prime
00 0100 xy: SECP256k1
00 1000 xy: RSA2048
00 1100 xy: Secret used for AES-128, HMAC-SHA-256, and HMAC-SHA-512
00 1110 xy: Secret seed used for BIP32

KeyParam
KeyParam identifies a subkey for a cryptographic algorithm.
For SECP256: 0 = public key; 1 = private key.
For RSA2048: 0 = Modulus; 1 = PublicExponent; 2 = P; 3 = Q; 4 = DP1; 5 = DQ1; 6 = PQ.
For Secret: KeyParam = 0.

BlobRnd
BlobRnd is a 12-byte random number generated by the TLS-SE application.

BlobID
BlobId is a 4-byte value set by the user of the TLS-SE application.

EncryptedValue
EncryptedValue is the result of AES-CCM encryption of a key value.
The authenticated data (AD), a 19-byte field, is defined as follows:
AD = BlobType (1B) || KeyIndex (1B) || KeyParam (1B) || BlobId (4B) || BlobRnd (12B)

Tag
Tag is the 16-byte AES-CCM tag value.

Blob Computation

A wrap key is computed as follows:

BlobPrefix = "export01"
Message (24B) = BlobPrefix (8B) || BlobId (4B) || BlobRnd (12B)
WrapKey = HMAC-SHA-256(Secret, Message)
Secret: up to 64 bytes

The authenticated data (AD), a 19-byte field, is defined as follows:
AD = BlobType (1B) || KeyIndex (1B) || KeyParam (1B) || BlobId (4B) || BlobRnd (12B)

The key value is encrypted using AES-CCM with the WrapKey.
EncryptedKey = AES-CCM(WrapKey, KeyValue)

TLS-SE App Commands, Version 3.03 (tls_se_303.cap)

Command	Description	G	Policy
?00	Version = 3.03	Y	5,01
?01[text]	Echo text	Y	5,02
?02	Disconnect (invalid command)	Y	always
?05[text]	Set root PSK identity	N	2,04
?06[text]	Set guest PSK identity	N	2,08
?0A	Get AppID (a SECP256k1 public key)	Y	2,10
?0B	Get app certificate: ECDSA (R, S) tuple, 64 bytes (128 hex digits)	Y	2,20
?0C[64 hex digits]	Authenticate (32-byte random number)	Y	2,40
?0E[128 hex digits]	Set certificate: ECDSA (R, S) tuple, 64 bytes (128 hex digits)	N	never
?30[128 hex digits]	Set secret for WrapKey #0: up to 128 hex digits (64 bytes)	N	3,01
?31[128 hex digits]	Set secret for WrapKey #1: up to 128 hex digits (64 bytes)	N	3,02
?33[14 hex digits]	Export key (14 hex digits, 7 bytes) BlobType (1B), KeyIndex (1B), KeyParam (1B), BlobId (4B) Return the blob value in hexadecimal, or ERROR	N	3,04
?34 [hex digits]	Import key BlobType (1B), KeyIndex (1B), KeyParam (1B), BlobId (4B) BlobRnd (12B), EncryptedValue, Tag (16B) Return OK or ERROR	N	3,08
?A0[64 hex digits]	Set PSK2 (used with the guest account); return the PSK2 value	N	4,01
?A1	Get PSK2 (used with the guest account)	N	4,04
?A2[64 hex digits]	Set PSK3 for the TLS-SE Binder and Derive procedures	N	4,02
?D0[64 hex digits]	Compute the Binder procedure using PSK3 and a 32-byte inputvalue	Y	4,10
?D1[64 hex digits]	Compute the Derive procedure using PSK3 and a 32-byte input value	Y	4,20
?D2[64 hex digits]	Compute the Derive procedure using PSK3 and a 32-byte input value Request an ACK	Y	4,40
?A5[2 hex digits]	Bit 0 (0x01): enable the guest PSK identity Bit 7 (0x80): check the server name	N	never
?A8[2 hex digits] [2 hex digits]	Set access policy First byte: block number; second byte: block value	N	never
?AA [OldPSK,NewPSK]	Set root PSK: OldPSK = 64 hex digits; NewPSK = 64 hex digits Return the NewPSK value	N	never
?FF[hex digits]	Echo a hexadecimal input value	Y	5,04
c[KeyIndex]	Clear Key KeyIndex= AlgoId || KeyId	N	0,01
C[KeyIndex]	Clear Key KeyIndex= AlgoId || KeyId, for AlgoId=0000 use SECP256P	N	0,01
g[KeyIndex]	Generate a key pair (SECP256P, SECP256k1, or RSA2048 only) KeyIndex= AlgoId || KeyId	N	0,02
G[keyIndex]	Generate a key pair (SECP256P, SECP256k1, or RSA2048 only) KeyIndex= AlgoId || KeyId, for AlgoId=0000 use SECP256P	N	0,02
*s[KeyIndex] [data]	Sign a value (up to 32 bytes); KeyIndex = Mode || AlgoId || KeyId SECP256P, SECP256k1, and RSA2048 only AlgoId may be 0000 for SECP256P and SECP256k1 Mode = b7b6; b7 = 0/1 (text/binary); b6 = 0/1 (single/multiple)	N	1,01
p[KeyIndex] [KeyParam]	Get public key; KeyIndex = AlgoId || KeyId SECP256P, SECP256k1, and RSA2048 only. AlgoId may be 0000 for SECP256P and SECP256k1 For RSA2048, KeyParam is mandatory, 0 = Modulus; 1 = PublicExpo	N	0,04

r[KeyIndex] [Optional KeyParam]	Get private key; KeyIndex = AlgoId || KeyId SECP256P, SECP256k1, and RSA2048 only AlgoId may be 0000 for SECP256P and SECP256k1 For RSA2048, KeyParam is mandatory: 2 = P; 3 = Q; 4 = DP1; 4 = DQ1; 5 = PQ	N	0,08
P[KeyIndex] [Optional KeyParam] [hex digits]	Set public key; KeyIndex = AlgoId || KeyId SECP256P, SECP256k1, and RSA2048 only AlgoId may be 0000 for SECP256P and SECP256k1 (65B key) For RSA2048, KeyParam is mandatory: 0 = Modulus (256B); 1 = PublicExpo (4B)	N	0,10
R[KeyIndex] [Optional KeyParam] [hex digits] X	Set private key; KeyIndex = AlgoId || KeyId SECP256P, SECP256k1, and RSA2048 only AlgoId may be 0000 for SECP256P and SECP256k1 (32B key) For RSA2048, KeyParam is mandatory: 2 = P; 3 = Q; 4 = DP1; 4 = DQ1; 5 = PQ	N	0,20
t[KeyIndex] [hex digits]	Set secret; KeyIndex = AlgoId || KeyId Secrets only (AlgoId = 1100 or 1110) Secrets are used by BIP32, HMAC-SHA-256, HMAC-SHA-512, or AES procedures	N	1,02
T[KeyIndex] [hex digits]	Set secret; KeyIndex = AlgoId || KeyId BIP32 secret only; up to 64 bytes (128 hex digits) AlgoId is ignored	N	1,02
b[KeyIndex] [Path hex digits] k	Compute a BIP32 tree; KeyIndex = AlgoId || KeyId AlgoId is ignored Path is a sequence of 32-bit values with the MSB set (hardened key)	N	1,04
v[KeyIndex]	Get secret; KeyIndex = AlgoId || KeyId AlgoId is ignored	N	1,08
*h[KeyIndex] [message]	Compute HMAC-SHA-256(secret, message) KeyIndex = Mode || AlgoId || KeyId Mode = b7b6; b7 = 0/1 (text/binary); b6 = 0/1 (single/multiple)	Y	0,80
*H[KeyIndex] [message]	Compute HMAC-SHA-512(secret, message) KeyIndex = Mode || AlgoId || KeyId Mode = b7b6; b7 = 0/1 (text/binary); b6 = 0/1 (single/multiple)	Y	0,80
*Axy[data]	AES encryption; KeyIndex = Mode || AlgoId || KeyId AES128(Secret, data); secret MUST be 16 bytes long AlgoId is ignored Mode = b7b6; b7 = 0/1 (text/binary); b6 = 0/1 (single/multiple)	Y	0,40
*axy[data]	AES decryption; KeyIndex = Mode || AlgoId || KeyId AES128(Secret, data); secret MUST be 16 bytes long AlgoId is ignored Mode = b7b6; b7 = 0/1 (text/binary); b6 = 0/1 (single/multiple)	Y	0,40
Z[RecIndex] [hex digits]	Write a value to record RecIndex 0...3: 1,280-byte maximum; 4...7: 64-byte maximum	N	1,10
Ixy	Read a value from record RecIndex	Y	1,20

* The s, h, H, A, and a commands use Special Operation Mode (SOM); see the next section.
P - A policy element consists of a block number (one byte) and a value (one byte) that identify a command.

Special Operation Mode (SOM)

For the s, h, H, a, and A commands, the KeyIndex parameter uses a dedicated format to support binary commands and/or multiple-input (pipeline) operations.

KeyIndex= Mode || AlgoId || KeyId
Mode= b_7b_6

- b_7 selects binary format instead of hexadecimal format.
  A binary command is encoded with a three-character prefix, a binary payload, and a two-character suffix (CR, LF).
  The response contains a binary payload and a two-character suffix (CR, LF).
  In the event of a fatal error, the connection is closed and no data is returned.
  An error MAY be indicated by an empty payload followed by a two-character suffix (CR, LF).
- b_6 indicates a multiple-input (pipeline) command. The payload is divided into 32-byte blocks for HMAC, 16-byte blocks for AES-128, and 32-byte blocks for ECDSA and RSA-2048 signatures. Each block is used as input to the corresponding operation.

In summary:

- KeyIndex = 00 || AlgoId || KeyId identifies a single command using a hexadecimal payload and key KeyId.
- KeyIndex = 10 || AlgoId || KeyId identifies a single command using a binary payload and key KeyId.
- KeyIndex = 01 || AlgoId || KeyId identifies a multiple-block command using a hexadecimal payload and key KeyId. HMAC uses 32-byte blocks, AES uses 16-byte blocks, and signatures use 32-byte blocks.
- KeyIndex = 11 || AlgoId || KeyId identifies a multiple-block command using a binary payload and key KeyId.

Access Policy

The root account has access to all commands.

For the guest account, access policy is controlled by six one-byte blocks. The root account can enable or disable individual commands using the ?A8[block#][value] command.

Block 0	b7	b6	b5	b4	b3	b2	b1	b0
cmd	h, H	a, A	R, X	P	r	p	g, G	c, C
Default: 0xC4	1	1	0	0	0	1	0	0

Block 1	b7	b6	b5	b4	b3	b2	b1	b0
cmd			I	Z	v	k, b	t, T	s
Default: 0x25	0	0	1	0	0	1	0	1

Block 2	b7	b6	b5	b4	b3	b2	b1	b0
cmd		?0C	?0B	?0A	?06	?05		
Default: 0x70	0	1	1	1	0	0	0	0

Block 3	b7	b6	b5	b4	b3	b2	b1	b0
cmd					?34	?33	?31	?30
Default: 0x00	0	0	0	0	0	0	0	0

Block 4	b7	b6	b5	b4	b3	b2	b1	b0
cmd		?D2	?D1	?D0		?A1	?A2	?A0
Default: 0x70		1	1	1	0	0	0	0

Block 5	b7	b6	b5	b4	b3	b2	b1	b0
cmd						?FF	?01	?00
Default: 0x07	0	0	0	0	0	1	1	1

TLS-SE Application Certification Procedure (ACP)

Upon instantiation, TLS-SE creates a public/private key pair on the SECP256k1 elliptic curve. The *Application Certification Procedure* (ACP) reads the public key and writes a certificate: an ECDSA signature of the public key consisting of two 32-byte values, R and S.

```
// GetID = Get Application Public Key (on the SECP256k1 elliptic curve)
?0A
043288117A7871F1CC92E3204D444BD9E656C2047D4FCE189F2F3F22AF01B07D2665F0C5332
06333E37454A8D00A2803E07BFF7356ED6AE74D94D874334A022AEF
// Set Certificate= ECDSA_CAPrivKey(SHA2(AppPubKey))= 64 bytes= R || S
?0E55D20B301E6E6A543B8FF2DA1F7C42371042A88A556CF4ECD0E76BF9740C51C5D0CF9741
2BA12B6A8640BA48A90D3B6CA18C87981D7E95E0B7D3FEDEE068D2CF
OK
```

TLS-SE Session Authentication Procedure (SAP)

The *Session Authentication Procedure* proves that the remote node knows the TLS-SE private key (TLS-SE-PrivKey) and the TLS handshake secret.

```
// GetID = Application Public Key
?0A
>>043288117A7871F1CC92E3204D444BD9E656C2047D4FCE189F2F3F22AF01B07D2665F0C53
3206333E37454A8D00A2803E07BFF7356ED6AE74D94D874334A022AEF
// Get TLS-SE Certificate
?0B
>>55D20B301E6E6A543B8FF2DA1F7C42371042A88A556CF4ECD0E76BF9740C51C5D0CF97412
BA12B6A8640BA48A90D3B6CA18C87981D7E95E0B7D3FEDEE068D2CF
// Authenticated Session Procedure
// Sign = Authenticate using a 32-byte random value
// return ECDSA_TLS-SE-PrivKey(SHA2(HandshakeSecret || Random))
?0C7F69B857C6C2675BC8D5238E3E8BFC4C633FB5E39DD07F4760F508084FD1B482
>>304402206D2E688731C2673F977BD49B37D6CEC966323E966E34DE426D424AC5506F4A4B0
2203F5462E3D0AA7A1ED410ADDB29AE7C980EFC0136028FDF8533843D6A3C854ABF
```

TRAIT Command

A trait command has an extra '**.**' character at the beginning of the three-character prefix.
A trait command returns a response after execution and then closes the TLS session.

A trait command can be used with OpenSSL to print the hexadecimal response payload, as shown below:
For Windows:
echo -ne ".command\r\n" | openssl s_client -quiet other_parameters… 2>nul
For Linux:
echo -ne ".command\r\n" | openssl s_client -quiet other_parameters 2>/dev/null

TLS-SE-IO

TLS-SE input/output is an OPTIONAL feature for Internet of Things (IoT) applications.

A TLS-SE-IO command has a '#' prefix.

It is exported in cleartext in a decrypted TLS record packet, with the first three bytes set to zero (PTCOL, Version_Major, Version_Minor).

The exported packet has the following format:
00 00 00 Length_MSB Length_LSB Output-Message TLS-Type=0x17

The input message, ending with TLS-Type = 0x17, is encoded in one or more RECV ISO 7816 APDUs according to the IETF TLS-SE-IO draft.

The Derive command requiring an ACK (?D2) requires the TLS-SE-IO feature. The output message is "ASKCrLf", and the input message MUST be "OKCrLf0x17" for further processing.